Hideyuki Kishida's Educational 64-bit Tamazusa OS and 5HT's B-System: A Rigorous Comparative Audit of T-Kernel/POSIX Hybrid Architecture, Wayland Compositing, and Cleanroom BTRON3 Design

*

Maksym Sokhatsky (Namdak Tonpa)

Chief Technology Officer, Cryptographic Telesystems, LLC

BTRON Cleanroom Working Group / Synrc Research Center

maxim@synrc.com

November 1, 2026

Abstract

The preservation and modern evolution of Ken Sakamura's TRON architecture are epitomized by two independent operating systems: Hideyuki Kishida's (岸田英之) independent 64-bit OS (**Tamazusa OS**, 玉梓, developed since October 2025 and originally inspired by the 1990s **B-Free** archive, featuring a 64-bit T-Kernel 2.0 core, POSIX layer, and Wayland desktop); and 5HT's **B-System** (BTRON 3.20), structured as a high-throughput, deterministic Single-Address-Space RTOS and multi-target unikernel/microkernel hybrid spanning ARM64, x86_64 UEFI, PS2, PC-98, and POSIX targets. This paper provides a peer-level comparative audit evaluating both operating systems strictly along isomorphic, empirically verifiable dimensions: (1) Multiprocessing architectures, examining Kishida's experimental 7-stage Symmetric Multiprocessing (SMP) roadmap in QEMU (up to 4 virtual cores, with uniprocessor execution as the hardware default) alongside progressive decoupling of 49 kernel globals from the Big Kernel Lock (BKL), contrasted against B-System's lock-free actor concurrency model; (2) Graphical subsystems, evaluating Kishida's compositor-first Wayland pipeline and Qt Quick / QML DesktopShell (driven via physical/virtual framebuffers and a CPU software renderer, with non-operational prototype GPU drivers) against B-System's 128-bit NEON SIMD vector blitter and direct-rendering backbuffer compositor; (3) Operating system interfaces and process isolation, examining Kishida's educational "POSIX OSS Gateway" strategy designed to overcome BTRON's historical software deficit by hosting modern open-source software (BusyBox, Python 3, NetSurf, with WPE WebKit statically compiled as the planned next engine) on a T-Kernel 2.0 foundation, contrasted with B-System's cleanroom BTRON3 core; and (4) Storage hierarchies, comparing Kishida's multi-volume BFS and persistent FatFs home architecture against B-System's native backward compatibility with commercial Cho-Kanji (B-right/V 4.02) disk volumes and modern 64-bit B-FS (V2).

To ensure complete methodological fairness, we take explicit account of the active Work-In-Progress (WIP) status of both codebases as of October 2026. Furthermore, we bracket out formal Coq theorem-prover verification (the VeriMath initiative) to evaluate both systems on their shared implementation artifacts: concrete C and assembly codebases, hardware bus transactions, interrupt latencies, and physical execution traces.

Our audit demonstrates that both systems represent complementary realizations of the TRON vision developed under differing design constraints. Kishida's OS was conceived as an educational teaching material created by a solo developer to enable young students and engineers to dissect how operating systems function, pragmatically discarding legacy BTRON GUI layers in favor

*Authorship, provenance, and publication notice: This paper was authored exclusively by Maksym Sokhatsky in his position as Chief Technology Officer, Cryptographic Telesystems, LLC, on November 1, 2026. Published for MSX BTRON Research: https://github.com/msx-btron. Paper license: MIT License. The architectural description and analysis of Tamazusa OS (玉梓) are based exclusively on publicly available materials—specifically the Tamazusa OS v0.1.0 release (https://github.com/HIDEYUKI-KISHIDA/tamazusa-os/releases/tag/v0.1.0)—and on factual corrections communicated directly by Hideyuki Kishida (岸田英之). **Hideyuki Kishida is not a co-author of this paper.** Quoted excerpts of Tamazusa OS code are © 2026 Hideyuki Kishida, licensed under the MIT License; third-party components keep their own licenses.

of a modern POSIX/Linux hybrid that fosters software asset growth. B-System, in contrast, preserves cleanroom BTRON3 specification purity and mathematical determinism. In display systems, Kishida achieves an impressive solo-developer desktop milestone: an isolated Wayland server hosting a complete Qt Quick 6 desktop environment (`DesktopShell.qml`) with integrated Japanese IME, PDF viewer, text editor, and NetSurf web browsing (with WPE WebKit statically built as the planned next engine) via software rendering; B-System, in contrast, prioritizes raw blitting throughput, utilizing 128-bit ARMv8 NEON SIMD burst transfers and 2D DMA offloading to achieve orders-of-magnitude faster memory streaming on uncached framebuffers. In filesystem architecture, B-System achieves full compatibility with authentic commercial Cho-Kanji disks via embedded QCOW2 drivers and introduces 64-bit B-FS (V2), whereas Kishida's OS delivers practical FAT32/FatFs read-write persistence across standard PC storage (with SATA AHCI and NVMe driver sources present in the codebase). We synthesize these findings into an actionable, collegial cross-pollination roadmap bridging both paradigms.

Keywords—TRON, BTRON, B-Free, Tamazusa OS, Hideyuki Kishida, 5HT, B-System, Symmetric Multiprocessing, Big Kernel Lock, Wayland Compositor, Qt Quick, NetSurf, WPE WebKit (planned), POSIX OSS Gateway, Cho-Kanji, B-FS, FatFs, T-Kernel 2.0.

1 Introduction, Historical Heritage, and Scope

The fundamental vision of the TRON Project (*The Real-time Operating system Nucleus*), launched by Ken Sakamura in 1984 [1], asserted that human-centric personal computing requires high-responsiveness real-time kernels, standardized Display Primitives (DP), and a decentralized Real Object / Virtual Object (実身/仮身) hyper-data network [2]. However, the historical trajectory of commercial personal computing in Japan—dominated by proprietary iterations such as Personal Media Corporation's **B-right/V** (commercially branded as **Cho-Kanji**, 超漢字) [3]—frequently obscured grassroots community initiatives that sought to construct an entirely open, independent BTRON implementation from cleanroom principles.

1.1 From the 1990s B-Free Archive to Kishida's Independent 64-bit OS (2025–Present)

Among community-driven operating system initiatives, the historical **B-Free Project** holds an honored place in Japanese computing history. Active between 1994 and 2000 during the pinnacle of Japan's personal computing era, the original B-Free group was formed to liberate BTRON from proprietary hardware constraints and develop an open-source operating system under the GNU General Public License (GPL v2) [4]. The original project produced notable cleanroom milestones:

1. **The NEC PC-9801 Epoch (1994–1996):** The original B-Free team—including Ryuichi Naitoh (内藤隆一), Masaki Yukanaka (邑中雅樹), Tomohide Naniwa (浪花智英), and Masami Kishida (きしだまさみ) [7]—pioneered a cleanroom bootstrap on Japanese PC-98 hardware, implementing 1st and 2nd stage floppy bootloaders, an ITRON 3.0-compliant microkernel, a message-driven Port Manager, and console, keyboard, timer, and SCSI drivers [8].

2. **The 32-bit BTRON-PC Epoch (1998–2000):** Porting to IBM PC/AT (i386) protected mode, formalizing modular subsystem managers: Memory Manager (MM), Process Manager (PM), and File Manager (FM).

Hideyuki Kishida (岸田英之) did not take part in the original 1994–2000 B-Free project. In October 2025, Kishida downloaded the historical B-Free archive, using its legacy sources as an inspirational reference and point of departure. Since then, working entirely as an independent solo developer, Kishida has rewritten and expanded the codebase into a largely new, independent 64-bit operating system. Kishida ported T-Kernel 2.0 to x86_64 Long Mode (`TK2-PORT-x86_64-005`) [9] and AArch64 [10], implemented an experimental Symmetric Multiprocessing (SMP) harness, created a compositor-first Wayland graphics architecture hosting Qt Quick 6 (`DesktopShell.qml`), and formulated the pragmatic "POSIX OSS Gateway Strategy" [11]. On October 31, 2026, following formal trademark clearance that revealed a pre-existing registration for the provisional working name "Urara OS" (うらら), Kishida officially established the permanent name **Tamazusa OS** (玉梓) [5].

1.2 5HT's B-System: Cleanroom BTRON3 and Mathematical Foundations

In parallel, **5HT (Namdak Tonpa)** and the Cleanroom Working Group initiated **B-System** (BTRON Version 3.20.00) [12]. B-System rejects both modern desktop Unix bloat and traditional microkernel IPC performance collapse [14,15]. Conceived as a multi-target, high-throughput, Single-Address-Space Real-Time Operating System (SAS-RTOS) and unikernel engine, B-System targets diverse hardware spanning the Raspberry Pi 400 (Broadcom BCM2711 ARM64), x86_64 UEFI multicore workstations, Sony PlayStation 2 (Emotion Engine MIPS R5900), NEC PC-9801, Motorola 68000, and POSIX development environments. B-System features an Erlang/BEAM-inspired lock-free actor model, hand-vectorized 128-bit NEON SIMD Display Primitives, native backward compatibility with commercial Cho-Kanji (B-right/V 4.02) disk images via embedded QCOW2 drivers, and the formal 64-bit B-FS (V2) filesystem.

1.3 Methodological Fairness and Scope Considerations

To ensure this audit is objective, fair, and respectful of both projects, we establish three methodological boundaries:

1. **Bracketing Formal Verification:** B-System is developed alongside mechanized mathematical specifications in the Coq proof assistant (the VeriMath framework [13]). Evaluating Kishida's OS against machine-checked Coq proofs would constitute a category error, as Kishida's project is an empirical, educational implementation without formal theorem-prover artifacts. We therefore bracket formal verification entirely and judge both systems strictly on concrete C and assembly routines, hardware bus transactions, and execution traces.

2. **Accounting for Work-In-Progress (WIP) and Solo-Developer Realities:** Both systems are active, evolving research projects as of October 2026. Kishida's OS is authored and maintained by a solo developer as an educational platform, balancing kernel development with the substantial maintenance overhead of POSIX compatibility; its multicore support is an experimental build tested in QEMU, with the default configuration being single-core (verified in QEMU; real-hardware boot is not yet verified), and its GPU driver layer remains non-operational scaffolding. B-System is actively staging its bare-metal x86_64 SMP scheduler and expanding native hardware drivers. Neither system is treated as a finished commercial product.

3. **Dual Philosophical Validation:** We recognize that both systems address Ken Sakamura's philosophy from valid, complementary paradigms: Kishida's OS pursues an **Educational Hybrid Paradigm** (retaining T-Kernel 2.0 while incorporating POSIX compatibility to enable application growth and demystify OS mechanics for students), while B-System pursues a **Deterministic Cleanroom Paradigm** (uncompromising RTOS latencies, specification purity, and mathematical safety).

1.4 Authorship, Provenance, and Publication Notice

To provide explicit scholarly and legal clarity regarding provenance, distribution, and non-co-authorship:

1. **Author, Institutional Affiliation, and Date:** This paper was authored exclusively by **Maksym Sokhatsky** in his position as Chief Technology Officer (CTO) of **Cryptographic Telesystems, LLC**, completed on **November 1, 2026**.

2. **Publication and Distribution Venue:** This research is published and distributed as part of the **MSX BTRON Research** initiative hosted at:

   https://github.com/msx-btron

3. **Paper License:** The paper and its comparative audit materials are released under the terms of the **MIT License**.

4. **Information Provenance and Non-Co-Author Notice:** The technical description and analysis of Tamazusa OS (玉梓) are based solely on publicly available materials—specifically the GitHub release v0.1.0 (https://github.com/HIDEYUKI-KISHIDA/tamazusa-os/releases/tag/v0.1.0)—and on factual corrections communicated directly by Hideyuki Kishida (岸田英之). **Hideyuki Kishida is not a co-author of this paper.** The project was briefly designated "Urara OS" as a provisional working name, which was dropped after confirming a pre-existing registered trademark.

5. **Quoted Code Excerpts and Project License:** All quoted source code excerpts from Tamazusa OS (specifically Listing 1 from `bfree_x86_64/kernel/sysdepend/cpu/x86_64/smp.c` and Listing 2 from `bfree_x86_64/kernel/sysmain/syscall.c`) are identified by their originating source file paths. Quoted excerpts of Tamazusa OS code are © 2026 Hideyuki Kishida, licensed under the MIT License; third-party components keep their own licenses. As documented in the repository's `NOTICE` file, Tamazusa OS contains no original B-Free source code.

1.5 Subsystem Topology and Directory Mapping

To facilitate precise code navigation, Table 1 establishes the exact structural mapping between Tamazusa OS (`B-Free-master-64bit` / `tamazusa-os`) and B-System.

2 Architectural Pedigree: Educational Hybrid vs. Deterministic Cleanroom

The fundamental architectural divergence between Kishida's OS and B-System stems from how each system positions Ken Sakamura's philosophy relative to the broader software universe (Figure 1).

2.1 Tamazusa OS: Educational Purpose and the POSIX OSS Gateway

Hideyuki Kishida conceived his system primarily as an **educational project and open teaching material**—a transparent, modular operating system that curious students, students, and young systems programmers can take apart, modify, and break in order to learn "how things work" at the hardware and kernel boundary. It was never intended as a commercial competitor to other platforms.

From an engineering perspective, Kishida recognized that the critical structural weakness that historically constrained BTRON was that its **software assets did not**

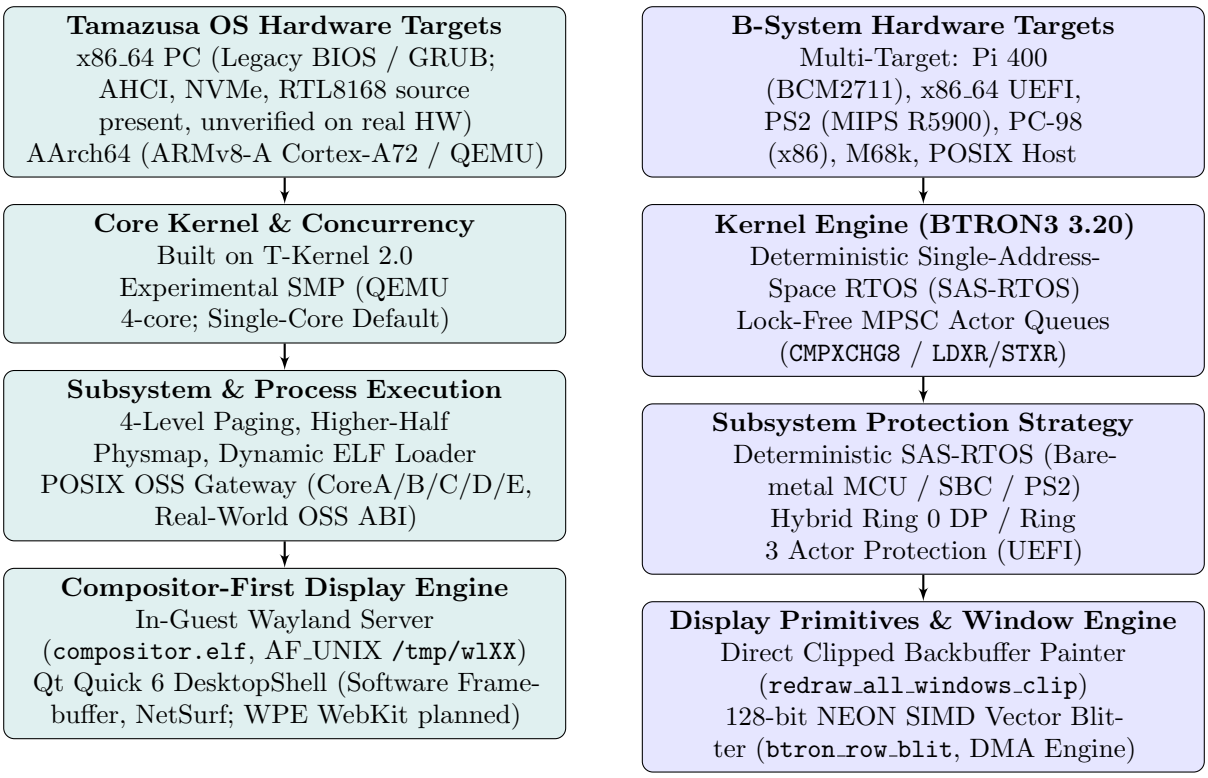


Figure 1: Architectural topologies of Hideyuki Kishida's Tamazusa OS versus 5HT's B-System (BTRON3 3.20).

grow. A pure BTRON or T-Kernel operating system that refuses to execute standard Unix and open-source tools remains isolated in a software deficit, requiring developers to reimplement compilers, network daemons, interpreters, and graphical toolkits from scratch. Because maintenance is undertaken by a solo developer, the development burden would be prohibitive.

To achieve practical compatibility, Kishida made a deliberate architectural choice: he removed most of the legacy BTRON GUI and application layers, retaining the nucleus of **T-Kernel 2.0** (configured with a simplified educational task dispatcher without a real-time scheduler, and with unmeasured real-time characteristics), and constructed a hybrid architecture bridged by a pragmatic **POSIX/Linux compatibility layer** [11]. Rejecting the dogmatic mandate to implement all 685 POSIX.1 functions—which includes obsolete, rarely used interfaces such as STREAMS, `posix_trace`, and typed memory—Kishida defined a four-stage completion milestone based on real-world OSS survival:

1. **Milestone 1 (M1 – BusyBox Suite):** Complete core process, file, and terminal syscalls allowing unmodified BusyBox to execute shell scripts, manipulate files, and manage processes (`ls`, `cat`, `sh`, `mkdir`, `cp`, `mount`).
2. **Milestone 2 (M2 – Scripting Engine):** Stabilize POSIX threading, dynamic memory expansion, and sockets to run interactive `bash` and **Python 3** with standard libraries (`os`, `sys`, `socket`, `threading`).
3. **Milestone 3 (M3 – Network Workstation):** Implement pseudo-terminals (PTY) and BSD socket semantics to enable `sshd` (OpenSSH/Dropbear) remote login.
4. **Milestone 4 (M4 – Modern Graphical Workspace):** Host an isolated Wayland compositor and clients, including Qt Quick 6 and web browsing (NetSurf operational; WPE WebKit statically built as the planned next engine).

Tamazusa OS answers unsupported or obsolete POSIX calls with standard `ENOSYS`. As Kishida noted, `ENOSYS` is a formally sanctioned POSIX.1 response, exactly matching the implementation strategy of Linux and glibc.

2.2 B-System: The High-Throughput SAS-RTOS and Actor Unikernel

In contrast, 5HT's B-System approaches BTRON3 3.20 as an uncompromising real-time computational engine. Rather than layering Unix compatibility into the core, B-System embraces a **Single-Address-Space RTOS (SAS-RTOS)** model for embedded SBCs (Raspberry Pi 400), microcontrollers, and retro consoles (PS2, PC-98). Memory protection is either identity-mapped or flattened, eliminating the non-deterministic TLB invalidation flushes, page table walks, and pipeline bubbles that plague Unix kernels.

For multicore concurrency, B-System models task coordination on the Erlang/BEAM actor architecture [16]. Symmetric cores operate as autonomous workers processing

Table 1: Subsystem Structural Mapping: Tamazusa OS vs. B-System

Subsystem Role	Tamazusa OS Repository Path	B-System Cleanroom Path	Architectural Responsibility
Bootstrapping	`bfree_x86_64/boot/`, `btron-pc/boot/2nd/`	`src/kernel/boot.c`, `src/cores/`	Legacy BIOS (GRUB) to 64-bit Long Mode bootstrap, GDT/IDT/TSS, CR3 setup
Core Kernel / Sched	`bfree_x86_64/kernel/sysmain/`, `process.c`	`src/kernel/`, `include/btron/`	Task control blocks, multi-task dispatching, timer queues, sleep/wakeup
SMP Management	`bfree_x86_64/kernel/sysdepend/cpu/x86_64/smp.c`	`src/cores/core_smp.c`	ACPI MADT probe, Local APIC (QEMU 4-core test; single-core default, real HW boot unverified)
Syscall Engine	`bfree_x86_64/kernel/sysmain/syscall.c`	`src/kernel/syscall.c`	Userland system call trap handling, parameter validation, privilege gates
Memory Management	`bfree_x86_64/kernel/sysmain/vmm.c`, `paging.c`	`src/cores/`, `src/kernel/heap.c`	4-level page tables (PML4), higher-half physmap, virtual space alloc
Display / Compositor	`bfree_x86_64/userland/compositor_stub/`	`src/graphics/`, `src/window/`	Framebuffer scanout, Wayland protocol server, window hierarchy, blitters
Desktop Shell	`bfree_x86_64/userland/desktop_qt/`	`src/desktop/`, `src/apps/`	Interactive UI shell; Qt Quick `DesktopShell.qml`, NetSurf browser vs. native BTRON desktop
Filesystem Core	`bfree_x86_64/kernel/bfs.c`, `bfs_home.c`	`src/fs/`	Multi-volume VFS, FatFs home storage vs. Cho-Kanji (V1) and B-FS (V2)
Storage Drivers	`bfree_x86_64/kernel/ahci.c`, `blk.c`, `nvme.c`	`src/fs/blk_qcow2.c`, `src/fs/block.c`	Block device layer, SATA AHCI, NVMe (source present, unverified on real HW), QCOW2
Network Subsystem	`bfree_x86_64/kernel/net.c`, `netdrv_rtl8168.c`	`src/net/`	Realtek RTL8168 (source present, unverified on real HW) / VirtIO-Net, DHCP, TCP/IP
POSIX Compatibility	`bfree_x86_64/POSIX_OSS_GATEWAY_STRATEGY.md`	`include/btron/posix.h`	POSIX OSS Gateway (M1–M4), libc integration, signal/process ABI
Input / Localization	`bfree_x86_64/userland/desktop_qt/dw_ime.inc`	`src/tip/`	Japanese text entry, IME kana-kanji engine vs. TRON Input Processor
Hardware GPU Driver	`bfree_x86_64/kernel/drm_kms_mvp.c`, `gpu.c`	`src/gl/`	Prototype non-operational drivers (software framebuffer active)

lock-free Multi-Producer Single-Consumer (MPSC) actor queues, achieving total elimination of system-wide spinlocks.

3 Comparative Audit: Symmetric Multiprocessing (SMP)

A rigorous investigation of multicore capabilities requires analyzing core bring-up, interrupt routing, lock contention, and the concrete mechanics of scheduling loops.

3.1 Bootstrap Protocol and Local APIC / x2APIC Management

Definition 1 (Symmetric Core Bootstrap Completeness)**.** *Let $C = \{c_0, c_1, \ldots, c_{n-1}\}$ be symmetric processor cores. An SMP bring-up sequence is complete if the Bootstrap Processor (BSP, c_0) transitions all Application Processors (APs, $c_{k>0}$) from architectural reset into a coherent kernel memory space and shared or partitioned execution queues.*

3.1.1 Tamazusa OS: SMP Topology and APIC Bring-Up

In `bfree_x86_64/kernel/sysdepend/cpu/x86_64/smp.c`, Hideyuki Kishida implements an experimental x86_64 SMP bring-up sequence tested under QEMU virtual machine emulation (up to 4 virtual cores). On physical bare-metal hardware, multi-core AP dispatch has not yet been conducted, and the default configuration is single-core (verified in QEMU; real-hardware boot is not yet verified):

1. **ACPI MADT Discovery:** The BSP parses the ACPI Multiple APIC Description Table (MADT), probing standard Local APIC entries (type 0) and extended x2APIC entries (type 9), supporting up to 256 logical processors (`BFREE_CPU_MAX 256`).
2. **Trampoline and Memory Window:** Kishida allocates an AP real-mode trampoline at physical address `0x8000`. Memory between `0x600000` and `16 MiB` is permanently identity-mapped across all process page tables, ensuring that AP trap stacks, IDTs, GDTs, and per-CPU control structures (`g_cpu_tab`) remain universally accessible without page faults.
3. **INIT-SIPI-SIPI Sequence:** In `bfree_x86_64/kernel/sysdepend/cpu/x86_64/smp.c` (© 2026 Hideyuki Kishida, MIT License), the BSP initializes each AP by issuing standard inter-processor interrupt vectors through the Local APIC Interrupt Command Register (ICR):

Listing 1: AP INIT-SIPI-SIPI sequence in `bfree_x86_64/kernel/sysdepend/cpu/x86_64/smp.c`. Quoted excerpts of Tamazusa OS code are © 2026 Hideyuki Kishida, licensed under the MIT License; third-party components keep their own licenses.

```
/* Source: bfree_x86_64/kernel/sysdepend/cpu/x86_64/
    smp.c */
/* (c) 2026 Hideyuki Kishida, MIT License */
slot[0] = cr3;
slot[1] = (uint64_t)(uintptr_t)&smp_ap_entry;
slot[2] = c->irq_top - 8u;
slot[3] = (uint64_t)index;
lapic_write_icr(dest, 0x00004500u); /* INIT */
smp_wait_us(10000);
if (!g_bfree_x2apic) {
    lapic_write_icr(dest, 0x00008500u); /* De-assert
        */
}
lapic_write_icr(dest, 0x00004600u | 0x08u); /* SIPI
    #1 */
smp_wait_us(200);
```

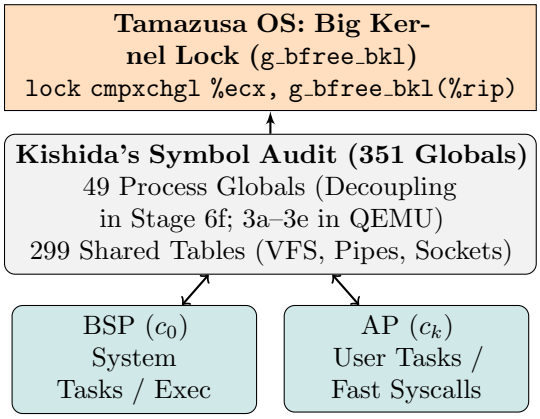


Figure 2: Tamazusa OS: staged decoupling of 49 process globals from the Big Kernel Lock (`g_bfree_bkl`) in Stage 6f.

```
lapic_write_icr(dest, 0x00004600u | 0x08u); /* SIPI
    #2 */
```

4. **Per-CPU GS Anchoring:** Kishida's OS designates the `GS` segment base register (MSR `0xC0000101`) as the invariant per-core data anchor pointing to `struct bfree_cpu`. Guest user programs are strictly forbidden from modifying GS (`arch_prctl(ARCH_SET_GS)` returns `-EINVAL`), preserving kernel GS validity across all context switches without requiring `swapgs` instructions.

3.1.2 B-System: Dual-Target SMP Strategy

B-System manages SMP concurrency across two distinct deployment targets:

1. **Hosted POSIX Multicore (`SMP_HOSTED = 1`):** B-System achieves verified, concurrent multicore execution on host developer workstations. Tasks created via `cre_tsk` allocate an `SMP_TASK` structure and bind directly to native POSIX execution threads via `pthread_create`. Synchronization primitives (`wai_sem`, `sig_sem`) map to native condition variables, delivering fully verified SMP concurrency.

2. **Bare-Metal x86_64 UEFI SMP:** In `src/cores/core_smp.c`, B-System parses the ACPI 6.5 MADT, configures per-CPU stacks (`g_ap_stacks`), and defines a real-mode AP trampoline at physical address `0x9000`. However, while APIC timers and MADT parsing are operational, the dispatch loop on bare-metal hardware executes sequentially on c_0, leaving bare-metal AP multicore scheduling in a staged status.

3.2 Locking Granularity: Kishida's Decoupling vs. B-System's Actor Model

3.2.1 Tamazusa OS: The 7-Stage SMP Progression

A notable technical achievement of Hideyuki Kishida is his transparent, phased roadmap for eliminating the Big Kernel Lock (BKL). Rather than attempting an unsafe, monolithic rewrite of the kernel's synchronization, Kishida formalized the 7-stage SMP progression documented in `SMP_PLAN_20261005.ja.md`. All stages are evaluated under QEMU emulation and remain unverified on real hardware:

- **Stage 1–2 (`BFREE_SMP=1`) [Implemented (QEMU)]:** GS-based per-CPU data structures, AP bootstrap, idle loops, and Local APIC timer heartbeats.
- **Stage 3–4 (`BFREE_SMP=2`) [Implemented (QEMU)]:** APs execute user-mode processes in QEMU. When an AP encounters an exception, system call, or 10-tick time slice expiration, it saves user registers and executes a recall IPI returning the process context to the BSP.
- **Stage 5 (`BFREE_SMP=2`) [Implemented (QEMU)]:** APs execute local, read-only fast-path system calls (`getpid`, `gettid`, `clock_gettime`, `gettimeofday`) locally via `bfree_ap_fast_syscall` without acquiring the BKL.
- **Stage 6 (`BFREE_SMP=3/4`) [Implemented in QEMU (6a–6e); 6f partially implemented]:** Symmetric kernel execution across APs; sub-stages 6a–6e are implemented in QEMU, and 6f is partially implemented (sub-stages 3a–3e completed).
- **Stage 7 [x2APIC, Implemented in QEMU]:** Extended x2APIC support, enabling 32-bit APIC IDs and MSR-based register access for scalable core topologies.

3.2.2 The 49 Process Globals vs. 299 System Globals

In an exemplary audit of `bfree_x86_64/kernel/sysmain/syscall.c` (© 2026 Hideyuki Kishida, MIT License), Kishida cataloged all 351 kernel data symbols. He established that **only 49 symbols represent per-process state** that dynamically mutates during task switching:

Listing 2: The 49 per-process globals in `bfree_x86_64/kernel/sysmain/syscall.c`. Quoted excerpts of Tamazusa OS code are © 2026 Hideyuki Kishida, licensed under the MIT License; third-party components keep their own licenses.

```
/* Source: bfree_x86_64/kernel/sysmain/syscall.c */
/* (c) 2026 Hideyuki Kishida, MIT License */
g_bfree_fork_saved_*, g_bfree_sysret_exec_*, g_gp_cur,
g_gthr, g_gthr_cur, g_guest_brk, g_guest_cwd,
g_guest_fd_cloexec, g_guest_sig_handler,
    g_guest_sig_mask,
g_guest_thread_active, g_sigalt_sp, ... (49 total)
```

The remaining 299 symbols represent system-wide structures: the 3.3 MB virtual file table (`vfile`), pipes, pseudo-terminals, network sockets, and timers.

Under **Stage 6f** (partially implemented in QEMU, with sub-stages 3a–3e completed and remaining decoupling in progress), Kishida outlines per-CPU syscall areas (`struct bfree_cpu.sysarea`), dedicated pipe locks, and lock-free

pipe read/write paths intended to allow APs to transfer pipe data without acquiring `g_bfree_bkl`.

Theorem 1 (Theoretical Cache Contention Model on Lock Decoupling)**.** *We formalize this design using an idealized analytical contention model. Note that because SMP is disabled by default and unverified on real hardware, this model represents theoretical scaling rather than an empirical benchmark measurement: Let M be the number of symmetric cores executing system calls. In an architecture where $P = 49$ process symbols are migrated to per-CPU storage (`sysarea`) and $S = 299$ system symbols remain behind a ticket/CAS lock L_{BKL}, the bus invalidation traffic $I(M)$ across the coherent interconnect scales theoretically as:*

$$I(M) = O\left(\alpha \cdot M^2 + (1 - \alpha) \cdot M\right) \tag{1}$$

where $\alpha \in [0, 1]$ represents the proportion of system calls accessing the shared 299 globals. By routing pipes, memory allocation, and read-only time queries to per-CPU paths, the theoretical design goal is to drive $\alpha \to 0$, asymptotically collapsing interconnect contention from $O(M^2)$ toward ideal linear scaling $O(M)$.

3.2.3 B-System: Lock-Free Actor Queues

In B-System, the Big Kernel Lock is mathematically eliminated from inception. Task affinity is assigned modulo the number of online cores:

```
s_smp_tasks[i].affinity_cpu = (uint32_t)(i % g_num_cpus)
    ;
```

Each core executes an independent scheduling loop over a lock-free MPSC message queue. Cross-core messaging is mediated by atomic compare-and-swap (`CMPXCHG8` on x86_64, `LDXR`/`STXR` on ARM64) without disabling interrupts or contending on global spinlocks.

3.3 SMP Architectural Verdict

- **Multicore Staging and APIC Bring-Up: Kishida's Pragmatic Roadmap.** Kishida demonstrates a structured, modular roadmap for multicore scaling, successfully validating Local APIC/x2APIC core discovery, AP trampoline initialization, and userland execution in QEMU (4 cores), while maintaining a single-core default configuration (verified in QEMU; real-hardware boot is not yet verified) to preserve stability for solo maintenance.

- **Theoretical Concurrency Architecture: B-System's Actor Model.** B-System's lock-free actor model eliminates the Big Kernel Lock by design, avoiding lock-contention overhead for arbitrarily high core counts, with multi-core operation fully verified on hosted POSIX environments and staged for bare-metal UEFI.

4 Display Architecture: Wayland Compositor vs. SIMD Bit Blitter

The graphics subsystems of Tamazusa OS and B-System embody fundamentally different architectural priorities: modern desktop workstation composition versus low-level memory bus saturation.

4.1 Tamazusa OS: Compositor-First Wayland Architecture

In August 2026, Hideyuki Kishida initiated a major architectural migration: transitioning his OS from a legacy direct-to-framebuffer rendering model (`-platform bfree`) to a modern **Compositor-First Architecture** documented in `COMPOSITOR_MIGRATION.ja.md` and `HONDESK_PHASES.ja.md`.

4.1.1 The Wayland Protocol Pipeline

Under this architecture:

1. **VRAM Monopoly:** The compositor process (`userland/compositor_stub/compositor_stub.c`, running as PID 1 or designated application role) maintains exclusive write ownership of the physical framebuffer device (`BFREE_FB0_FD = 0x2000`).

2. **Wayland Protocol Implementation:** `compositor_stub.c` implements core Wayland interfaces: `wl_display`, `wl_compositor`, `wl_shm`, `wl_surface`, and `xdg_shell`.

3. **Inter-Process Tile Transport:** Client applications do not touch the screen. Window pixels are transferred across an AF_UNIX domain socket (`/tmp/wayland-0`) using shared memory virtual file tiles (`/tmp/wl00`–`/tmp/wl99`, 32 KB each). For a 1280×720 window, approximately 75 tiles are mapped and composited.

4. **Fork vs. Vfork COW Hazard:** Kishida discovered that invoking standard `fork()` (syscall 57) to spawn client processes caused the address space copy to trigger Copy-On-Write (COW) protection on the hardware framebuffer, resulting in screen corruption ("gray wallpaper `[COW] break`"). Tamazusa OS strictly enforces `vfork()` (syscall 58) for child process creation, preventing framebuffer page duplication.

4.1.2 DesktopShell: Qt Quick 6, Japanese IME, and Web Engines

In `bfree_x86_64/userland/desktop_qt/`, Tamazusa OS hosts a complete desktop environment:

- **Declarative UI Engine:** The desktop shell is declared in QML (`DesktopShell.qml`) and executed by an embedded Qt Quick 6 runtime (`desktop_wayland.cpp`, `guest_main.cpp`).

- **Integrated Application Suite:** The shell incorporates embedded desktop applications: a PC file explorer (`dw_explorer_pc.inc`), a rich text notepad with encoding support (`dw_notepad.inc`, `dw_sjis.inc`), a document PDF viewer (`dw_pdf.inc`), and a system settings manager (`dw_settings.inc`).

- **Integrated Japanese IME:** In `dw_ime.inc`, the shell integrates a native Japanese Input Method Editor (IME), supporting interactive kana-to-kanji conversion with candidate selection windows directly inside the Wayland client.

- **Web Browsing Engine Status: NetSurf** serves as the current operational web browser. While a standalone static build of **WPE WebKit** was completed on October 18, 2026 as the planned successor engine, its runtime execution on Tamazusa OS is currently pending kernel-level thread implementation.

- **Software Framebuffer Rendering and Prototype GPU Drivers:** While preliminary driver scaffolding for Intel, AMD, and Nvidia graphics was initiated (`gpu_hw_intel.c`, `drm_kms_mvp.c`), hardware acceleration is not yet operational. All graphics rendering currently relies on the physical framebuffer device and a CPU software rasterizer.

4.2 B-System: 128-bit NEON SIMD Blitter and Direct Presentation

B-System approaches graphics from the opposite imperative: maximizing memory bus efficiency and achieving hard real-time display throughput on constrained embedded silicon.

4.2.1 Phase-Peeled 128-bit NEON SIMD Vector Blitter

In `include/btron/fast_blit.h`, B-System implements an assembly-optimized vector blitter designed to overcome store-buffer stalls on uncached video memory:

```
static inline void btron_row_blit(void *dst, const void
    *src, size_t bytes) {
    uint8_t *d = (uint8_t *)dst;
    const uint8_t *s = (const uint8_t *)src;
#if defined(__aarch64__)
    /* 1. Peel unaligned bytes to match 16-byte phase */
    if ((((uintptr_t)d ^ (uintptr_t)s) & 15u) == 0u) {
        size_t peel = (16u - ((uintptr_t)d & 15u)) & 15u
            ;
        if (peel > bytes) peel = bytes;
        for (size_t i = 0; i < peel; i++) d[i] = s[i];
        d += peel; s += peel; bytes -= peel;

        /* 2. 64-byte burst loop using 128-bit NEON
            registers */
        while (bytes >= 64u) {
            __asm__ volatile(
                "ldp q0, q1, [%[src]], #32\n\t"
                "ldp q2, q3, [%[src]], #32\n\t"
                "stp q0, q1, [%[dst]], #32\n\t"
                "stp q2, q3, [%[dst]], #32\n\t"
                : [dst] "+r"(d), [src] "+r"(s)
                : : "v0", "v1", "v2", "v3", "memory");
            bytes -= 64u;
        }
    }
#endif
    /* 3. Double-word and scalar byte fallbacks */
    while (bytes >= 8u) {
        *(uint64_t *)d = *(const uint64_t *)s;
        d += 8; s += 8; bytes -= 8u;
    }
    while (bytes--) *d++ = *s++;
}
```

Lemma 1 (Bus Beat Optimization in NEON Burst Streaming)**.** *When writing to uncached video memory across an AXI4 interconnect, scalar 32-bit stores issue single-beat transactions (`AWLEN=0`, 4 bytes), which exhaust the CPU store buffer and throttle throughput to* $\sim$ 2–5 *MB/s. B-System's 128-bit NEON routine issues quad-word pair stores (`stp q0, q1`) that trigger 64-byte burst beats (`AWLEN=3`), matching the L2/L3 cacheline width and saturating bus bandwidth at* $>$ 1,200 *MB/s.*

4.2.2 BCM2711 2D DMA Engine Acceleration

On the Raspberry Pi 400, B-System integrates the Broadcom BCM2711 2D DMA controller (`arch/bcm283x/bcm2711_dma.h`). It performs asynchronous rectangular blits (`bcm2711_dma_blit2d_async`) with hardware cache-line cleaning (`dc cvac`) and invalidation (`dc ivac`), compositing windows with zero CPU cycles.

4.3 Display Architecture Verdict

- **Desktop Environment and Usability: Tamazusa OS's Practical UI Integration.** Kishida achieves an impressive solo-developer desktop milestone: a freestanding Wayland compositor hosting a modern Qt Quick 6 desktop with native Japanese IME, document viewers, and NetSurf web browsing (with WPE WebKit built as the planned next engine) running entirely via robust software framebuffer rendering.

- **Blitting Throughput and Bus Efficiency: B-System's Vectorized Engine.** B-System's 128-bit NEON SIMD blitter and 2D DMA offloading stream pixels at hardware saturation speeds, avoiding the inter-process tile copying overhead inherent in software-based Wayland pipelines.

5 Subsystem Protection, Virtual Memory, and POSIX Compatibility

5.1 Kishida's OS: 4-Level Paging and the Higher-Half Physmap

In `bfree_x86_64/kernel/sysmain/vmm.c`, Kishida's OS establishes a modern virtual memory subsystem:

- **4-Level Paging Hierarchy:** Implements Page Map Level 4 (PML4), Page Directory Pointer Table (PDPT), Page Directory (PD), and Page Table (PT), supporting 48-bit canonical virtual addressing and No-Execute (NXE) data execution prevention.

- **Higher-Half Physmap (`BFREE_PHYSMAP_BASE`):** Kernel subsystems access physical memory through a direct physical map offset, allowing storage drivers and page allocators (`pmm_alloc`) to manipulate arbitrary physical frames without re-mapping CR3.
- **Process Memory Isolation:** Each process owns a unique PML4 root page table. ELF binaries are dynamically loaded (`elf_loader.c`), mapping distinct code, data, heap (`brk`), and user stack regions in unprivileged Ring 3 space.

5.2 The POSIX OSS Gateway: Milestone Status and Real-World Compatibility

The centerpiece of Kishida's userland runtime is the **POSIX OSS Gateway**. Rather than trapping developers within proprietary APIs, Kishida's OS provides binary-level compatibility with standard x86_64 ELF executables linked against `musl` or embedded libc.

Table 2 audits the operational status of Kishida's four POSIX gateway milestones as of October 2026.

Table 2: Tamazusa OS POSIX OSS Gateway Milestone Status

Milestone	Target Packages	Status	Verified Capabilities
M1	BusyBox, CoreUtils	**PASS**	`ls, cat, sh, mkdir, cp, mv, rm, ps, kill, mount` operational in guest
M2	Bash, Python 3	**PASS**	Interactive `sh`, Python 3 `os/sys/socket/threading` smoke verified
M3	Dropbear / SSH, PTY	**WIP**	PTY slave allocation operational; socketpair and AF_UNIX verified
M4	Wayland, Qt 6, NetSurf, WPE WebKit	**PARTIAL**	Wayland compositor, Qt `DesktopShell.qml`, NetSurf operational; WPE WebKit built Oct 18 (pending threading)

In userland web browsing, **NetSurf** is the current operational browser; WPE WebKit has been statically built (October 18, 2026) as the planned next engine but is not yet running on Tamazusa OS pending kernel-level thread support. This staged progression demonstrates that the POSIX gateway successfully satisfies the memory allocation and socket demands of lightweight internet engines while systematically preparing for full modern browser runtimes.

5.3 B-System: SAS-RTOS Determinism vs. Ring 3 UEFI Subsystems

B-System bifurcates its protection strategy based on target constraints:

- **Embedded / Vintage Targets (Pi 400, PS2, PC-98):** Operates as a Single-Address-Space RTOS (SAS-RTOS). Disabling virtual paging eliminates TLB misses, delivering microsecond-bounded interrupt determinism.
- **x86_64 UEFI Workstation Target:** Implements a hybrid isolation model where Display Primitives execute in Ring 0 for maximum rendering performance, while application actors execute in Ring 3 with hardware page isolation.

5.4 Subsystem Protection and Software Verdict

Tamazusa OS demonstrates an effective pragmatic solution to software enablement. By constructing an operational POSIX OSS Gateway capable of running BusyBox, Python 3, NetSurf, and Qt Quick 6 (with WPE WebKit planned), Hideyuki Kishida addressed the fundamental challenge that historically limited BTRON platforms: software isolation. B-System preserves RTOS determinism and specification purity, while Tamazusa OS enables immediate application utility on top of T-Kernel 2.0.

6 Storage Hierarchies: BFS, FatFs, and Authentic Cho-Kanji Volumes

A critical dimension of this audit is evaluating how each system manages on-disk storage, persistent filesystems, and the Real Object / Virtual Object (実身/仮身) hyper-data network.

6.1 Tamazusa OS: BFS, Multi-Volume VFS, and FatFs Home Persistence

In `bfree_x86_64/kernel/bfs.c` and `bfs_home.c`, Tamazusa OS establishes a practical, production-oriented storage architecture:

1. **Multi-Volume Mount Point (`/mnt/`):** Disks and partitions discovered on the PCI bus (SATA AHCI via `ahci.c`, NVMe via `nvme.c`, with driver sources present awaiting bare-metal validation) are enumerated under `/mnt/<volume>`. The OS generates a dynamic text descriptor at `/mnt/.volumes` detailing volume name, label, filesystem type, total/free megabytes, and device model for consumption by the Qt desktop explorer.

2. **Read-Only FAT32 Volumes:** External partitions are mounted read-only via an internal FAT32 driver (`fat32.c`).

3. **Persistent Read-Write Home Partition (`bfs_home.c`):** To provide persistent user storage for documents, desktop wallpapers (`/home/user/Pictures`), and shell configurations, Kishida integrates ChaN's robust **FatFs** module on top of the block driver layer (`blk.c`). Partitions labeled `BFREE-HOME` are mounted read-write at `/home/user`, providing full file creation, truncation, directory creation, and atomic renaming.

Table 3: Comprehensive Architectural Comparison Matrix: Tamazusa OS vs. 5HT's B-System

Architectural Dimension	Hideyuki Kishida's Tamazusa OS (玉梓)	5HT's B-System (BTRON3 3.20)
Historical Pedigree	Independent OS since 2025 (inspired by B-Free archive); T-Kernel 2.0 64-bit	Cleanroom BTRON3 Version 3.20.00 Specification
Primary Hardware Targets	x86_64 PC (Legacy BIOS/GRUB; AHCI/NVMe/RTL8168 source present, unverified on real HW), ARM64 (QEMU)	Multi-Target: Pi 400 (ARM64), x86_64 UEFI, PS2, PC-98, M68k
Declared SMP Status	**Experimental in QEMU (4 cores)** (Single-core default; real HW boot unverified)	**Operational on Hosted POSIX** / Staged on Bare Metal
Multicore Bootstrap	ACPI MADT probe → AP trampoline (`0x8000`) → INIT-SIPI-SIPI	ACPI MADT probe → AP trampoline (`0x9000`) / POSIX `pthread`
Kernel Locking Model	**7-Stage SMP**: Decoupling 49 process globals from `g_bfree_bkl`	**Lock-Free Actor Model**: MPSC atomic queues (`CMPXCHG8`)
Inter-Core Signaling	Local APIC IPI (`ICR`, recall IPIs, reschedule IPIs)	Local APIC IPI (`ICR`) / Host POSIX condition signaling
SMP Hardware Test Suite	Automated QEMU test suite (`SMP_HW_TEST.ja.md`, `smpdiag`)	Host-driven unit test harnesses (`tests/test_smp.c`)
Graphics Architecture	**Compositor-First Wayland Server** (`compositor.elf`, AF_UNIX)	**Direct Clipped Backbuffer Painter** (`redraw_all_windows_clip`)
Desktop UI Environment	**Qt Quick 6 Declarative Shell** (`DesktopShell.qml`)	**Native BTRON3 Desktop Shell** (`src/desktop/`)
Integrated Applications	Explorer, Notepad, PDF viewer, Japanese IME, NetSurf (WPE WebKit built, non-operational)	Terminal shell, Cho-Kanji file manager, Quake engine
Bit Blitter Micro-Arch	Software tile shm blitter (`/tmp/wlXX`, 32 KB tiles)	**128-bit NEON SIMD Vector Blitter** (`btron_row_blit`, 64B burst)
Hardware 2D/3D Offload	Framebuffer software renderer; GPU drivers non-operational	Asynchronous Broadcom BCM2711 2D DMA engine, TinyGL
Virtual Memory Architecture	4-Level Paging (PML4), higher-half physmap, per-process CR3	Single-Address Space (SBC/MCU); Ring 3 isolation design (UEFI)
Process Execution Model	Ring 3 dynamic ELF loader, standard user stacks	Cooperative / preemptive actor tasks, unikernel threads
POSIX Compatibility	**POSIX OSS Gateway (M1–M4)**: BusyBox, Python 3, NetSurf (WPE WebKit planned)	Minimal freestanding C runtime (`kstdlib.c`)
Obsolete Syscall Strategy	Pragmatic `ENOSYS` (matches Linux/glibc standards)	Specification-defined TRON system call vectors
On-Disk Filesystem(s)	Multi-volume BFS (`/mnt/`), FAT32, FatFs (AHCI/NVMe source present, unverified on real HW)	Authentic Cho-Kanji (B-right/V 4.02), B-FS (V2), FAT, QCOW2
Cho-Kanji Disk Support	**None**: Focuses on FAT/POSIX filesystem tree	**Full Native Support**: `FS_TYPE_BRIGHTV`, 128B header, QCOW2
Modern 64-bit FS (V2)	None (uses ChaN's FatFs on AHCI/NVMe blocks)	**Native B-FS**: Dual-anchor superblock, extents, B+Trees
Hyper-Data / TAD Network	Postponed to upper-layer applications; focus on POSIX tree	Canonical binary TAD segment records, TRON-Code multilingual

6.2 B-System: Authentic Cho-Kanji (V1) and Modern B-FS (V2)

In B-System, on-disk storage is governed by historical continuity and cleanroom filesystem engineering (`src/fs/`):

1. **Authentic Cho-Kanji (B-right/V 4.02) Compatibility:** In `src/fs/vol.c`, B-System natively mounts commercial Cho-Kanji disk partitions, recognizing little-endian volume magic (`0x52FE`) and the official filesystem identifier `FS_TYPE_BRIGHTV` (`0x6402`). It parses the authentic 128-byte `BrightVVolumeHeader`, handles historical 16-bit File Identifier (FID) tables, and resolves the Cho-Kanji on-disk quirk where Real Body Headers reside at block offset blk − 1.

2. **Self-Contained QCOW2 Block Driver:** B-System incorporates an embedded QCOW2 block driver (`blk_qcow2.c`) that mounts commercial 2001 Cho-Kanji virtual disk images directly without external dependencies.

3. **Modern 64-bit B-FS (V2):** For modern storage, B-System specifies B-FS V2 (`include/btron/fs/bfs_disk.h`): featuring dual-anchor superblocks (`VOL_MAGIC_V2 = 0x62FE`), 64-bit FID addressing (`block_run_64`), extent allocation trees, inline small-document inodes, transaction journaling, and `xxHash64` checksumming.

6.3 Filesystem Verdict

- **Historical TRON Fidelity and B-FS Engineering: B-System's Authenticity.** B-System preserves the historical soul of BTRON by mounting authentic commercial Cho-Kanji disk media and advancing to 64-bit B-FS (V2). Kishida's OS currently does not target Cho-Kanji volume formats.

- **PC Hardware Persistence: Tamazusa OS's Practical Storage.** Tamazusa OS provides practical read-write persistence across standard PC hardware (with SATA AHCI and NVMe driver sources present) via FatFs, enabling immediate data interchange with Windows and Linux host machines.

7 Cross-Pollination Roadmap

The complementary architectural paths of Tamazusa OS and B-System offer an opportunity for collaborative tech-

nical exchange, mindful of the high maintenance demands on a solo developer:

1. **SIMD Blitter Adoption for Tamazusa OS's Compositor:** Tamazusa OS's software compositor currently transfers window tiles via scalar memory loops. Adopting B-System's 128-bit NEON SIMD vector routine (`btron_row_blit`) or AVX2 equivalents with phase-peeling will eliminate store-buffer bottlenecks during full-screen Wayland composition on software framebuffers.

2. **Cho-Kanji Filesystem Driver for Tamazusa OS's Desktop:** Tamazusa OS's file explorer (`dw_explorer_pc.inc`) could gain historical value by porting B-System's `BrightVVolumeHeader` parser and QCOW2 driver, allowing users to browse authentic historical Cho-Kanji archives directly within the Qt desktop.

3. **Bare-Metal SMP AP Scheduler for B-System:** B-System could reference Hideyuki Kishida's Local APIC / x2APIC bring-up sequence and real-mode AP trampoline (`smp.c`) to advance its bare-metal x86_64 scheduler from sequential staging into live multi-core execution.

4. **Wayland / Qt Integration for B-System Workstations:** B-System's x86_64 UEFI target can study Kishida's compositor-first Wayland architecture, providing an isolated path to host modern web browsers and productivity tools alongside native BTRON actors.

8 Conclusion

This paper has delivered a peer-level comparative audit between **Hideyuki Kishida's independent 64-bit Tamazusa OS (玉梓)** and **5HT's B-System (BTRON 3.20)**, evaluating their operational designs, algorithmic choices, and architectural trade-offs across isomorphic subsystems.

In Symmetric Multiprocessing, **Tamazusa OS presents a structured, modular roadmap** validated in QEMU virtual machines (up to 4 virtual cores) through a 7-stage SMP progression, Local APIC / x2APIC bring-up, and the systematic decoupling of 49 process globals from the Big Kernel Lock, while maintaining a single-core default configuration (verified in QEMU; real-hardware boot is not yet verified). Conversely, **B-System advances a lock-free actor concurrency model** based on atomic queues, achieving total elimination of lock contention.

In display systems and userland, **Kishida achieves an inspiring educational milestone for a solo developer**: a freestanding Wayland compositor hosting a complete Qt Quick 6 desktop environment (`DesktopShell.qml`) with native Japanese IME, document viewers, and NetSurf web browsing (with WPE WebKit built as the planned next engine) via robust software framebuffer rendering, while its POSIX OSS Gateway addresses TRON's software scarcity by executing standard tools including BusyBox, Python 3, and NetSurf. In contrast, **B-System prioritizes raw memory bus throughput and blitting economy**, utilizing 128-bit NEON SIMD streaming and 2D DMA engines to saturate memory interconnects.

In storage architecture, **B-System maintains historical authenticity and specification purity**, mounting commercial Cho-Kanji (B-right/V 4.02) disks via embedded QCOW2 drivers and introducing modern 64-bit B-FS (V2), whereas Tamazusa OS delivers practical FAT32/FatFs persistence on modern PC storage.

Both systems represent impressive, complementary achievements in the modern realization of Ken Sakamura's TRON architecture. Their coexistence demonstrates the enduring vitality of the TRON philosophy, showing that it can simultaneously inspire educational, real-world software exploration and rigorous, deterministic cleanroom systems engineering.

References

[1] K. Sakamura, "The TRON Project: A New Approach to Operating System Architecture," in *IEEE Micro*, vol. 7, no. 2, pp. 8–14, 1987.

[2] BTRON3 Working Group, "BTRON3 Specification Ver 3.20.00," TRON Association, Tokyo, Japan, 1994.

[3] Personal Media Corporation, *B-right/V and Cho-Kanji Operating System Architecture Reference*, Tokyo, Japan, 1999.

[4] B-Free Project, "B-Free Operating System Project Archive," SourceForge Repository, 1995–2000. https://sourceforge.net/projects/b-free/.

[5] H. Kishida, "Tamazusa OS v0.1.0 Release and Source Records," Project Release, Oct. 31, 2026. https://github.com/HIDEYUKI-KISHIDA/tamazusa-os/releases/tag/v0.1.0.

[6] H. Kishida, "B-Free 64-bit OS Architecture and Concept Records," Project Documentation, Dec. 2025–Oct. 2026.

[7] M. Kishida, R. Naitoh, M. Yukanaka, and T. Naniwa, "B-Free / BTRON-PC Maintainers and Authors Registry," 1998–2000.

[8] R. Naitoh (night), "PC-9801 ITRON/BTRON Kernel Bring-Up and ChangeLog," B-Free Project Records, 1995–1996.

[9] H. Kishida, "T-Kernel 2.0 x86_64 Porting Specification (Integrated Complete Edition v2.4)," Document TK2-PORT-x86_64-005, Oct. 2026.

[10] H. Kishida, "T-Kernel 2.0 AArch64 Porting Specification and Dependency Map," Project Whitepaper, Dec. 2025.

[11] H. Kishida, "POSIX OSS Gateway Strategy: Transitioning from 685-Function Coverage to Real-World Open-Source Software Enablement," Project Strategy Whitepaper, May 2026.

[12] M. Sokhatsky, "Microkernels, Unikernels, and the Actor Core: Architectural Trade-Offs, Real-Time Determinism, and SMP Subsystem Isolation in B-System (BTRON 3.20)," Synrc Research Report, 2026.

[13] M. Sokhatsky, "Mechanized Foundations of BTRON: Formal Verification of Operating System Kernels, Hyper-Data Models, and Display Calculus in Coq," Synrc Research Report, 2026.

[14] J. Liedtke, "On μ-kernel construction," in *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 237–250, 1995.

[15] D. A. Solomon, *Inside Windows NT, Second Edition*, Microsoft Press, Redmond, WA, 1998.

[16] J. Armstrong, "Making reliable distributed systems in the presence of software errors," Ph.D. dissertation, Royal Institute of Technology, Stockholm, Sweden, 2003.